TECHNICAL PROPOSAL

Papyrus v3.0 Claude Code-like TUI Architecture

A comprehensive proposal for building a fully-featured, Claude Code-like interactive terminal application with streaming markdown rendering, diff visualization, multi-provider LLM support, and the Papyrus orchestration engine.

Based on: pi-mono (badlogic) + claude-canvas (dvdsgl)

Stack: TypeScript + Bun + Ink (React for Terminal)

Timeline: 8 weeks, 4 phases

Target: Claude Code / Qwen Code CLI parity

```
> Build a todo app in React
GATE All checks passed
SNAPSHOT Saved (a1b2c3d)
PLANNER Generating plan...
AUDIT PASS - 0 violations
BUILDER Applying 4 steps...
DONE in 8.2s. Ready to test.
```

Table of Contents

1. Executive Summary 4

2. Current State Analysis: Papyrus v2.0 5

2.1 Go Backend (cmd/papyrus/) 5

2.2 Node.js / Bun TUI (papyrus-tui/) 6

2.3 Critical Issues Identified 6

3. Reference Projects Analysis 7

3.1 pi-mono (badlogic/pi-mono) 7

3.2 claude-canvas (dvdsgl/claude-canvas) 7

3.3 Comparative Analysis and Design Decisions 8

4. Target Design: The Claude Code Experience 8

4.1 What Claude Code Looks Like 9

4.2 Papyrus v3.0 Screen Layout Specification 10

4.3 Interaction Model 11

5. Architecture and Technology Stack 11

5.1 Language and Runtime: TypeScript + Bun 11

5.2 Monorepo Structure 12

5.3 LLM Provider Architecture 13

5.4 Communication Architecture: MCP-Style Stdio 14

6. Detailed Component Design 14

6.1 Streaming Markdown Renderer 14

6.2 Diff Viewer 15

6.3 Tool Execution Visualization 15

6.4 Permission System 15

6.5 Session Management 16

6.6 Autocomplete System 16

7. Orchestration Engine: Porting from Go 17

7.1 Preserved Architecture: Plan-Audit-Build Pipeline 17

7.2 Enhanced State Machine 17

7.3 Auto-Heal Loop Enhancement 18

8. Implementation Roadmap 18

8.1 Phase 1: Foundation (Week 1-2) 18

8.2 Phase 2: Core Experience (Week 3-4) 19

8.3 Phase 3: Full Claude Code Parity (Week 5-6) 19

8.4 Phase 4: Polish and Distribution (Week 7-8) 20

9. Migration Strategy from v2.0 20

9.1 What We Keep 21

9.2 What We Discard 21

9.3 Configuration Compatibility 22

10. Risk Assessment and Mitigations 22

11. Dependencies 23

12. Conclusion 23

1. Executive Summary

This proposal presents a comprehensive redesign of Papyrus, transforming it from a basic command-line tool with a rudimentary terminal interface into a fully-featured, Claude Code-like interactive terminal application. The current Papyrus v2.0 implements a Go backend with a solid orchestration pipeline (Gate, Planner, Auditor, Builder) but suffers from a fragmented user experience: three separate TUI implementations (Go backend with Cobra, Node.js TUI, and Bun/Ink React TUI), a hardcoded API key exposed in source code, inconsistent port configurations, and a lack of the polished, immersive terminal experience that modern AI coding agents like Claude Code, Qwen Code, and DeepSeek Coder provide.

Our analysis of two key open-source projects, pi-mono by Mario Zechner (a production-grade AI agent toolkit with a custom ANSI/CSI TUI framework supporting 20+ LLM providers) and claude-canvas by David Siegel (an Ink-based TUI plugin for Claude Code that renders interactive components in tmux split panes via Unix domain socket IPC), has informed our design. We will adopt the best architectural patterns from both: the differential rendering and component-based architecture from pi-tui, and the React/Ink component model with tmux integration from claude-canvas, while building on the existing Papyrus orchestration engine (the Plan-Audit-Build pipeline, state machine, and auto-healing loop) which represents genuine intellectual value.

The target deliverable is a single TypeScript monorepo that compiles to a standalone binary via Bun, providing an immersive full-screen terminal experience with streaming markdown rendering, inline code diffing, tool execution visualization, session management, slash commands, file-aware autocomplete, and multi-provider LLM support. The architecture follows the Elm/TEA pattern with a strict unidirectional data flow, ensuring predictable state management even under the non-deterministic conditions inherent to LLM interactions. The result will be a tool that looks, feels, and behaves identically to Claude Code CLI, but powered by Papyrus's superior orchestration engine underneath.

2. Current State Analysis: Papyrus v2.0

Before designing the new system, we must thoroughly understand what currently exists, what works, what is broken, and what intellectual property from the existing codebase is worth preserving. The following analysis covers every source file in the Papyrus v2.0 project, evaluating each component for reuse potential.

2.1 Go Backend (cmd/papyrus/)

The Go backend consists of twelve command files registered under a Cobra root command. The backend implements a complete orchestration pipeline with five distinct phases: Gate (environment checks), Planner (LLM-powered plan generation), Auditor (deterministic rule engine), Builder (safe file patching), and a State Machine that persists workflow state to disk. The architecture is fundamentally sound and represents the core intellectual value of the project.

Command	File	Purpose	Reuse Verdict
root	main.go	Cobra root command with logger init	Rebuild in TypeScript
chat	chat.go	Interactive REPL with /plan, /run, /build commands	Rebuild with full TUI
run	run.go	Full pipeline: Gate + Snapshot + Plan + Audit + Build + Auto-heal	Core logic port to TS
plan	plan.go	Headless plan generation with dry-run support	Port to TS SDK function
init	init.go	Project scaffolding: .papyrus/, papyrus.yaml, TARGET.md	Port to TS CLI command
doctor	doctor.go	Health check + Sentinel validation + auto-repair	Port to TS CLI command
model	model.go	Provider switching (ollama/openrouter/demo)	Rebuild with multi-provider UI
rpc	rpc.go	JSON-RPC 2.0 WebSocket server on :9090	Rebuild with stdio IPC
rollback	rollback.go	Git snapshot restoration	Port to TS CLI command
optimize	optimize.go	Coach-Planner-Builder-Evaluator loop	Port to TS SDK function
mascot	mascot.go	Animated pixel art (Thoth/Khepri/Snowy)	Rebuild as TUI easter egg
bench	bench.go	12-task benchmark suite with evaluation	Port to TS SDK function
demo	demo.go	Pipeline test with model listing	Remove in v3
hello	hello.go	Connection test command	Remove in v3

Table 1. Go Backend Command Inventory and Reuse Assessment

2.2 Node.js / Bun TUI (papyrus-tui/)

The existing TUI is a single App.tsx file using Ink (React for the terminal) with a hardcoded OpenRouter API key, no session persistence, no plan/audit/build pipeline integration, and a minimal feature set limited to basic chat, file read/write, code execution, and directory listing. While it demonstrates the viability of Ink as a TUI framework, it is essentially a proof-of-concept chat client that does not leverage any of Papyrus's orchestration capabilities. The Nord color theme and layout structure (header, status bar, message area, input prompt) provide a useful visual reference but the implementation must be completely replaced.

2.3 Critical Issues Identified

Severity	Issue	Location	Impact
Critical	Hardcoded OpenRouter API key in source	papyrus.yaml, App.tsx	Security: key exposed in VCS
Critical	Missing go.mod / go.sum files	Project root	Non-compilable Go backend
High	Port mismatch: server :9090 vs client :9876	rpc.go, client.ts	RPC communication broken
High	Three duplicate TUI implementations	Go REPL, Node.js, Bun/Ink	Maintenance nightmare
High	No streaming LLM responses	All LLM calls	User waits for complete response
Medium	Builder does not validate search_block existence	builder package	Silent data corruption risk
Medium	No markdown rendering in output	All display paths	Poor readability for code blocks
Medium	State machine not resilient to crash	state package	Lost context on Ctrl+C
Low	Mascot animation is static (no animation loop)	mascot.go	Cosmetic only

Table 2. Critical and High-Issues Inventory from Code Review

3. Reference Projects Analysis

3.1 pi-mono (badlogic/pi-mono)

Pi-mono is a production-grade AI agent toolkit by Mario Zechner at version 0.67.68, organized as a seven-package TypeScript monorepo. It provides a unified LLM API supporting 20+ providers, a stateful agent runtime with event-driven architecture, and a custom-built TUI framework (pi-tui) that renders directly using ANSI/CSI escape codes without any third-party TUI library. The pi-tui framework achieves flicker-free rendering through CSI 2026 synchronized output, differential rendering with three update strategies, and a component-based interface where each component implements a simple contract: render(width) returns string arrays, and handleInput(data) processes keyboard events.

Key architectural patterns we will adopt from pi-mono include: the differential rendering engine that compares previous and current screen state to minimize terminal writes; the component lifecycle model with focus management and overlay positioning; the theme system with customizable styling interfaces; and the inline image rendering protocol support for Kitty and iTerm2 terminals. We will not adopt pi-mono's monorepo structure (seven packages is over-engineered for our scope) or its Go-based TypeScript compiler (tsgo), as Bun's native TypeScript execution is sufficient for our needs.

3.2 claude-canvas (dvdsgl/claude-canvas)

Claude-canvas is a proof-of-concept plugin for Claude Code that extends Claude with interactive terminal interfaces rendered as React/Ink components in tmux split panes. It communicates between Claude and canvas components via Unix domain sockets using a simple IPC protocol with typed messages (ready, selected, cancelled, update, close, getSelection, getContent). The plugin architecture allows Claude to spawn visual interfaces (calendars, documents, flight booking UIs) in separate tmux panes and receive user selections back through the IPC channel.

Key patterns we will adopt from claude-canvas include: the Ink/React component model for building interactive terminal widgets (which provides a rich ecosystem of hooks and familiar React patterns); the Unix domain socket IPC mechanism for bidirectional communication between the agent process and UI components; and the tmux integration for spawning auxiliary panes (enabling side-by-side views of code and plan visualization). We will not adopt claude-canvas's dependency on Bun-only APIs (Bun.connect, Bun.file) since we want Node.js compatibility, nor its limited canvas type system (only calendar, document, flight) which is too narrow for our needs.

3.3 Comparative Analysis and Design Decisions

Aspect	pi-mono	claude-canvas	Papyrus v3.0 Decision
TUI Framework	Custom ANSI/CSI (no library)	Ink v6 (React)	Ink v6 + custom ANSI for perf
Language	TypeScript (ESM)	TypeScript (Bun only)	TypeScript (Bun + Node.js)
Rendering	CSI 2026 + differential	Ink internal	Ink + CSI 2026 sync output
LLM Support	20+ providers unified	None (Claude Code)	5+ providers with streaming
IPC	RPC mode (stdio)	Unix domain sockets	Stdio JSON-RPC (MCP-style)
Session Mgmt	Branching, export, compaction	None	Full session with branching
Extensibility	Extensions, Skills, Themes, Packages	Skills, Canvas types	Plugins + Skills + Themes
Distribution	npm + Bun binary	Claude Code plugin	Standalone binary (Bun compile)
Build System	tsgo (Go TS compiler)	None (Bun runs TS)	Bun native (no build step)

Table 3. Comparative Architecture Decision Matrix

4. Target Design: The Claude Code Experience

4.1 What Claude Code Looks Like

Claude Code CLI presents a specific, recognizable terminal experience that has become the de facto standard for AI coding agents. When a user launches Claude Code, they see a full-screen terminal application (not line-by-line output) with several distinctive visual characteristics. The interface is dark-themed with carefully chosen contrast ratios that reduce eye strain during extended coding sessions. The input area sits at the bottom of the screen with a blinking cursor and a subtle prompt indicator. Above it, the conversation scrolls upward in a message thread where user messages are visually distinct from assistant messages through color and alignment.

The most distinctive feature is the streaming output. When Claude responds, text appears character-by-character (or word-by-word) rather than waiting for the complete response. Markdown is rendered in real-time: headers get their proper weight, code blocks are syntax-highlighted with proper background coloring, lists are indented correctly, and bold/italic text uses terminal escape sequences. When Claude decides to use a tool (read a file, write code, run a command), the tool invocation is displayed as a collapsible block with the tool name, input parameters, and output. The user can press Enter at any time during streaming to interrupt and redirect the conversation.

Additional UX patterns include: slash commands (/help, /model, /clear, /compact, /config) shown as an autocomplete dropdown when the user types "/"; file path autocomplete triggered by the @ symbol; a permission prompt system that asks for approval before file writes or command execution (unless auto-approve is configured); a diff viewer that shows added/removed lines in green/red when files are modified; and a status indicator showing token usage and cost tracking.

4.2 Papyrus v3.0 Screen Layout Specification

The Papyrus v3.0 interface will replicate the Claude Code layout with the following screen regions, ordered from top to bottom. Each region serves a specific purpose and is managed by an independent React/Ink component that can be updated independently through the differential rendering system.

Region	Position	Content	Component
Header Bar	Top (1 line)	App name, model indicator, session ID, token counter	StatusBar
Message Thread	Middle (scrollable)	User/assistant messages with streaming markdown	MessageList
Tool Blocks	Within thread	Collapsible tool invocations with input/output	ToolBlock
Diff Viewer	Within thread	Side-by-side or unified diff of file changes	DiffViewer
Permission Prompt	Above input	Approve/deny prompt for file writes and commands	PermissionPrompt
Input Area	Bottom (2-3 lines)	Text input with cursor, @ autocomplete, / commands	InputArea
Status Bar	Bottom (1 line)	Phase indicator (Gate/Plan/Audit/Build), spinner	StatusBar

Table 4. Screen Layout Region Specification

4.3 Interaction Model

The interaction model follows a conversation-first paradigm where every action flows through the message thread. Unlike v2.0 where the user had to type slash commands to trigger plan generation or build execution, v3.0 will use natural language understanding to detect intent. When the user types "build me a todo app in React", the system automatically triggers the full pipeline (Gate, Snapshot, Plan, Audit, Build) while displaying progress in real-time within the message thread. The pipeline phases are shown as status updates with spinners, and the final output (created files, applied patches, test results) is rendered as a structured summary card.

For power users who want explicit control, all pipeline commands remain available as slash commands: /plan to generate a plan without building, /audit to run the auditor, /build to execute from a saved plan, /rollback to restore the Git snapshot, and /status to show the current pipeline state. Session management commands include /compact (summarize conversation to reduce token usage), /branch (fork the conversation for exploration), /model (switch LLM provider), and /config (edit papyrus.yaml settings). The Escape key toggles between input mode and command mode (Vim-style), and Ctrl+C interrupts any running operation cleanly.

5. Architecture and Technology Stack

5.1 Language and Runtime: TypeScript + Bun

We are moving from Go to TypeScript as the primary implementation language. This decision is driven by several factors specific to our requirements. First, the TUI framework we have chosen (Ink, which is React for the terminal) is a JavaScript/TypeScript library, and building the TUI in a different language would require complex inter-process communication between the Go backend and the TypeScript frontend, introducing latency and complexity. Second, the LLM SDK ecosystem is overwhelmingly JavaScript-first: the Anthropic SDK, OpenAI SDK, and streaming SSE parsers are all native to JavaScript/TypeScript. Third, Bun's ability to compile TypeScript to a standalone binary eliminates the Go advantage of single-binary distribution.

Bun serves as both our runtime and our build tool. It executes TypeScript files directly without a compilation step during development (dramatically speeding up the development cycle) and compiles to a standalone binary for production distribution via the --compile flag. The binary is self-contained, requires no Node.js installation on the target machine, and runs identically on Linux, macOS, and Windows. Bun also provides native performance advantages for I/O-heavy operations (file system access, HTTP requests, child process spawning) which are the core operations of an AI coding agent.

5.2 Monorepo Structure

The project is organized as a single TypeScript monorepo with three packages. This is simpler than pi-mono's seven-package structure but provides clear separation of concerns. The papyrus package contains the TUI application and CLI entry point. The papyrus-engine package contains the orchestration pipeline (Gate, Planner, Auditor, Builder, State Machine) as a pure TypeScript library with no UI dependencies, making it independently testable and reusable. The papyrus-tui package contains all terminal UI components built on Ink.

```
papyrus-v3/
package.json
tsconfig.json
bun.lock
packages/
papyrus/ # CLI entry point + app orchestration
src/
index.ts # Main entry (CLI arg parsing)
app.tsx # Root Ink component
commands/ # Slash command handlers
hooks/ # Custom React hooks (useLLM, useSession)
package.json
papyrus-engine/ # Core orchestration (no UI deps)
src/
gate.ts # Environment checks
planner.ts # LLM-powered plan generation
auditor.ts # Deterministic rule engine
builder.ts # Safe file patching
state.ts # State machine + disk persistence
healer.ts # Auto-heal loop
llm/
provider.ts # Abstract provider interface
openrouter.ts # OpenRouter implementation
anthropic.ts # Anthropic Claude implementation
openai.ts # OpenAI implementation
ollama.ts # Ollama local implementation
streaming.ts # SSE streaming parser
tools/
read.ts # File read tool
write.ts # File write tool
edit.ts # File edit (search/replace) tool
bash.ts # Command execution tool
glob.ts # File pattern matching tool
grep.ts # Content search tool
package.json
papyrus-tui/ # Terminal UI components
src/
components/
App.tsx # Root layout component
Header.tsx # Top status bar
MessageList.tsx # Scrollable message thread
MessageBubble.tsx # User/assistant message renderer
Markdown.tsx # Streaming markdown renderer
```

```
CodeBlock.tsx # Syntax-highlighted code blocks
DiffViewer.tsx # File diff display (green/red)
ToolBlock.tsx # Collapsible tool invocation
InputArea.tsx # Bottom input with autocomplete
PermissionPrompt.tsx # Approve/deny dialog
Spinner.tsx # Loading indicator
StatusBar.tsx # Bottom status line
hooks/
useInput.ts # Keyboard input handling
useTheme.ts # Theme management
useAutocomplete.ts # File/@/command autocomplete
theme/
dark.ts # Dark theme (default, Claude-style)
light.ts # Light theme
nord.ts # Nord theme (from v2)
package.json
```

5.3 LLM Provider Architecture

The LLM provider system follows a unified interface pattern inspired by pi-mono's @mariozechner/pi-ai package. All providers implement a common Provider interface with methods for chat completion (with streaming support), tool call handling, token counting, and model listing. The streaming implementation uses Server-Sent Events (SSE) parsing with typed event objects that distinguish between text deltas, tool call starts, tool call deltas, thinking deltas, and usage information. This typed streaming approach allows the TUI to render different content types differently as they arrive: text goes to the markdown renderer, tool calls get their own collapsible blocks, and thinking content can be shown in a collapsed "reasoning" section.

Provider	Models	Streaming	Tool Calling	Auth Method
OpenRouter	Qwen, Claude, GPT-4, Llama, Mistral	SSE	Yes	API Key
Anthropic	Claude 3.5 Sonnet, Claude 3 Opus	SSE	Yes	API Key / OAuth
OpenAI	GPT-4o, GPT-4-turbo, o1	SSE	Yes	API Key
Ollama	Llama 3, Qwen, CodeLlama (local)	SSE	Yes	None (local)
Google	Gemini Pro, Gemini Flash	SSE	Yes	API Key

Table 5. Supported LLM Providers

5.4 Communication Architecture: MCP-Style Stdio

We adopt a Model Context Protocol (MCP) style stdio communication pattern for all inter-process communication. This means the Papyrus engine communicates with LLM providers by spawning them as child processes and exchanging JSON-RPC messages over stdin/stdout, rather than making HTTP calls. This approach has several advantages: it works seamlessly with Claude Desktop, Ollama, and any MCP-compatible tool server; it avoids HTTP server configuration issues (no port conflicts, no CORS problems); and it provides natural sandboxing since child processes inherit the parent's environment and permissions.

The stdio protocol defines three message types: request (client to server), response (server to client), and notification (either direction, no response expected). For LLM interactions, the client sends a "complete" request with the conversation messages and receives a streaming response via server-sent events on stdout. For tool execution, the engine sends a "tools/call" request and receives the tool result. This bidirectional streaming enables the real-time display of LLM thinking, tool execution progress, and final output that defines the Claude Code experience.

6. Detailed Component Design

6.1 Streaming Markdown Renderer

The streaming markdown renderer is the single most important visual component in the entire application. It is responsible for converting the incoming token stream from the LLM into properly formatted terminal output in real-time. Unlike static markdown rendering (where the full text is available upfront), streaming rendering must handle partial input gracefully: a code fence that has been opened but not yet closed, a list item whose children are still arriving, or a table row that is mid-parse. The renderer maintains an internal state machine that tracks the current parsing context (paragraph, heading, code block, list, table) and incrementally updates the display as new tokens arrive.

The rendering pipeline processes tokens through four stages. First, the tokenizer breaks the incoming text into markdown blocks (paragraphs, headings, code fences, lists, tables, blockquotes). Second, the block parser processes each block to identify inline elements (bold, italic, code spans, links). Third, the theme applicator maps semantic elements to terminal escape sequences using the active theme (colors, bold, underline, background). Fourth, the layout engine wraps text to the terminal width, handles word-breaking for long lines, and computes the vertical space required for each block. The output is an array of strings (one per terminal line) that the differential renderer compares against the previous frame to compute minimal updates.

6.2 Diff Viewer

When the Builder modifies files, the diff viewer displays the changes in a unified diff format with color coding: removed lines are shown with a red background, added lines with a green background, and unchanged context lines with the default background. Line numbers are shown on both sides. For large diffs (more than 50 lines), the viewer collapses unchanged regions with a "N lines hidden" indicator. The diff viewer also supports a side-by-side mode (activated by pressing "s" when the diff is focused) that shows the old and new versions in adjacent columns. This component is critical for the user to verify what the Builder has done before accepting changes.

6.3 Tool Execution Visualization

Every tool invocation (file read, file write, bash command, grep search) is displayed as a collapsible block within the message thread. Each tool block has a header showing the tool name, a status indicator (running/success/error), and the execution duration. When expanded, the block shows the input parameters and the output/result. For bash commands, the output is syntax-highlighted and truncated to 100 lines with a "show more" toggle. For file writes, the block shows the file path and a mini-diff of what changed. The user can press Enter on a tool block to expand/collapse it, allowing them to focus on the conversation flow while having detailed tool output available on demand.

6.4 Permission System

Papyrus v3.0 implements a Claude Code-style permission system that protects the user's workspace from unintended modifications. Before any file write or bash command execution, the system pauses and displays a permission prompt showing exactly what will happen: the file path for writes, the command for bash, and the expected impact. The user can approve (Enter), deny (Escape), or always-allow for the current session (a key toggle). The permission state is managed by a PermissionManager that supports three modes: ask (prompt for every action), auto-approve (never prompt, like --dangerously-skip-permissions in Claude Code), and smart-approve (auto-approve reads and searches, prompt for writes and commands). The smart-approve mode is the default and provides the best balance of safety and velocity.

6.5 Session Management

Sessions are the persistence mechanism that allows users to resume work across terminal restarts. Each session is identified by a UUID and stored in the .papyrus/sessions/ directory as a JSONL file (one JSON object per line, representing a single message or system event). Sessions include the full conversation history, pipeline state, created file snapshots, and token usage statistics. The user can list sessions (/sessions), switch between sessions (/session switch ID), branch a session (/branch creates a fork for exploration without affecting the main conversation), and compact a session (/compact summarizes the conversation to reduce token count while preserving key decisions and context). Session branching is particularly valuable: when the user wants to explore an alternative approach, they can branch, experiment, and then either merge the branch back or discard it.

6.6 Autocomplete System

The autocomplete system provides contextual suggestions as the user types, triggered by three special prefixes. The "@" prefix triggers file path autocomplete: the system scans the current project directory and displays matching files and directories in a dropdown list, using fuzzy matching (similar to fzf) to handle partial matches. The "/" prefix triggers slash command autocomplete: a filtered list of available commands (/plan, /run, /build, /audit, /rollback, /model, /clear, /compact, /branch, /config, /status, /sessions, /quit) with short descriptions. The "#" prefix triggers conversation context references: the user can reference previous messages, plan steps, or file paths from earlier in the conversation. The autocomplete dropdown is rendered as an overlay positioned above the input area, with keyboard navigation (arrow keys + Enter to select) and real-time filtering as the user continues typing.

7. Orchestration Engine: Porting from Go

7.1 Preserved Architecture: Plan-Audit-Build Pipeline

The core orchestration pipeline from Papyrus v2.0 is preserved in its entirety and ported to TypeScript. This pipeline is the primary intellectual property of the project and the key differentiator from generic AI chat tools. The pipeline consists of four sequential phases, each with clear inputs, outputs, and error handling. The Gate phase checks for required tools (git, language runtimes, LLM access) and validates the papyrus.yaml configuration. The Planner phase sends the user's goal (enriched with project context from git diff, TARGET.md rules, and conversation history) to the LLM and receives a structured JSON plan with specific actions (CREATE_FILE, EDIT_FILE, RUN_COMMAND). The Auditor phase runs deterministic checks on the plan against the rules defined in papyrus.yaml using regex matching and string scanning, producing a PASS/WARN/BLOCK verdict. The Builder phase executes the approved plan using safe file patching with exact string matching, atomic file replacement, and post-patch linting.

7.2 Enhanced State Machine

The state machine is enhanced from v2.0 to support crash recovery and interrupt handling. In v2.0, a Ctrl+C during the build phase would leave orphaned state files and potentially corrupted working directory files. In v3.0, the state machine listens for SIGINT/SIGTERM signals and performs a clean shutdown: it saves the current phase and all accumulated data to the session file, rolls back any in-progress file modifications (using the pre-build Git snapshot), and updates the state to INTERRUPTED. When the user resumes the session, the state machine detects the INTERRUPTED state and offers to continue from where it left off, restart the failed phase, or abort entirely.

State	Description	Recovery Action
IDLE	No active pipeline	None needed
GATE	Running environment checks	Re-run checks
PLANNING	LLM generating plan	Resume with existing context
AUDITING	Checking rules	Re-run audit with saved plan
BUILDING	Applying patches	Resume from last successful step
HEALING	Auto-fix loop active	Resume with error context
DONE	Pipeline completed	None needed
FAILED	Unrecoverable error	Manual intervention needed
INTERRUPTED	User cancelled or crash	Offer resume/restart/abort

Table 6. State Machine States and Recovery Actions

7.3 Auto-Heal Loop Enhancement

The auto-heal loop from v2.0 is preserved and enhanced with better diagnostics and smarter retry strategies. When the Builder fails (search block not found, lint error, runtime error from test command), the system enters healing mode. In v2.0, healing simply re-sent the error to the Planner. In v3.0, healing follows a three-tier strategy. Tier 1 (Quick Fix): for simple issues like missing imports or syntax errors, the system sends only the error message and the failing file to the LLM with a "fix this specific error" prompt, rather than re-planning the entire goal. Tier 2 (Targeted Replan): if the quick fix fails, the system identifies which plan step failed and asks the LLM to regenerate only that step, preserving the successful steps. Tier 3 (Full Replan): if targeted replanning also fails, the system falls back to generating a completely new plan, incorporating the error context and the lessons learned from the failed attempt.

8. Implementation Roadmap

The implementation is organized into four phases, each delivering a working (if minimal) product that can be tested and iterated upon. This phased approach ensures that we never have a broken codebase: each phase builds on a functioning foundation from the previous phase.

8.1 Phase 1: Foundation (Week 1-2)

Task	Description	Deliverable
Monorepo setup	Initialize Bun workspace with 3 packages	Buildable skeleton
papyrus-engine: Gate	Port environment checks to TypeScript	gate.ts with provider detection
papyrus-engine: State	Port state machine with crash recovery	state.ts with JSON persistence
papyrus-engine: LLM base	Abstract provider interface + OpenRouter impl	provider.ts + openrouter.ts
papyrus-tui: Shell	Ink app with dark theme, input area, message list	App.tsx renders "hello world"
papyrus-tui: Markdown	Basic markdown rendering (headings, code, bold)	Markdown.tsx with static render
CI/CD	GitHub Actions for lint + test + build	Automated checks on push

Table 7. Phase 1 Tasks

8.2 Phase 2: Core Experience (Week 3-4)

Task	Description	Deliverable
Streaming renderer	Real-time token-by-token markdown rendering	Markdown.tsx with streaming
papyrus-engine: Planner	Port plan generation with structured JSON output	planner.ts with plan schema
papyrus-engine: Auditor	Port deterministic rule engine	auditor.ts with regex + string checks
papyrus-engine: Builder	Port safe file patching with exact match	builder.ts with atomic writes
Tool visualization	Collapsible tool blocks in message thread	ToolBlock.tsx component
Permission system	Approve/deny prompts for writes and commands	PermissionPrompt.tsx
Slash commands	/plan, /run, /build, /clear, /quit	Command handler registry

Table 8. Phase 2 Tasks

8.3 Phase 3: Full Claude Code Parity (Week 5-6)

Task	Description	Deliverable
Diff viewer	Unified diff with green/red coloring	DiffViewer.tsx with collapse
Auto-heal loop	3-tier healing (quick fix, targeted, full replan)	healer.ts with tier selection
Session management	Persist/resume/branch/compact sessions	session.ts + SessionManager
Autocomplete	@ file paths, / commands, # context refs	useAutocomplete.ts hook
Multi-provider	Add Anthropic, OpenAI, Ollama providers	anthropic.ts, openai.ts, ollama.ts
Theme system	Dark (default), Light, Nord themes	Theme switcher + 3 theme files
Status bar	Token counter, cost tracking, phase indicator	StatusBar.tsx + useTokenCounter

Table 9. Phase 3 Tasks

8.4 Phase 4: Polish and Distribution (Week 7-8)

Task	Description	Deliverable
Binary compilation	Bun --compile for Linux, macOS, Windows	Standalone binary releases
Inline images	Kitty/iTerm2 protocol for image display	Image component in markdown
tmux integration	Split pane for plan/code side-by-side view	tmux spawn commands
Keyboard config	Customizable keybindings (Vim/Emacs defaults)	KeybindingManager
Benchmark suite	Port 12-task benchmark to TypeScript	bench.ts with scoring
Plugin system	Loadable plugins (like Claude Code extensions)	PluginManager + API
Documentation	README, CONTRIBUTING, API docs	Complete documentation site

Table 10. Phase 4 Tasks

9. Migration Strategy from v2.0

The migration from Papyrus v2.0 to v3.0 is a complete rewrite of the frontend (TUI) and a port of the backend (orchestration engine) from Go to TypeScript. There is no incremental migration path between the two because the fundamental technology stack changes from Go+Cobra+Node.js/Ink to TypeScript+Bun+Ink. However, the intellectual property (pipeline architecture, state machine design, rule engine logic, plan schema, auto-heal strategy) transfers directly. The migration strategy is therefore: preserve the design, rewrite the implementation.

9.1 What We Keep

The following elements from v2.0 are preserved in v3.0 with minimal modification. The papyrus.yaml configuration schema (LLM provider settings, project configuration, rules engine, workflow parameters) remains identical, ensuring backward compatibility for existing users. The TARGET.md concept (a human-readable rules file that the auditor enforces) is preserved and extended with support for YAML comments and section headers. The Plan JSON schema (goal, steps with actions CREATE_FILE/EDIT_FILE/RUN_COMMAND, dependencies_added field) is preserved exactly, allowing plans generated by v2.0 to be executed by v3.0. The state machine states (GATE, PLANNING, AUDITING, BUILDING, DONE, FAILED) are preserved with the addition of INTERRUPTED and HEALING states. The .papyrus/ directory structure (state.json, context directory, logs directory) is preserved and extended with a sessions/ subdirectory.

9.2 What We Discard

The following elements from v2.0 are removed entirely. The three separate TUI implementations (Go CLI REPL, Node.js TUI, Bun/Ink TUI) are replaced by a single Ink-based TUI. The gRPC/JSON-RPC WebSocket server (rpc.go) is replaced by MCP-style stdio communication, eliminating the port mismatch issue entirely. The mascot command (mascot.go) is removed as a non-essential feature (it may return as a TUI easter egg in a future version). The demo command (demo.go) and hello command (hello.go) are removed as development artifacts that have no place in a production tool. The entire internal Go package structure (internal/logger, internal/lock, internal/sentinel, internal/triage, internal/waiter, internal/optimizer, internal/bench, internal/git) is reimplemented in TypeScript within the papyrus-engine package.

9.3 Configuration Compatibility

Papyrus v3.0 reads the same papyrus.yaml format as v2.0, with the following additions. The LLM section gains a thinking field (none/low/medium/high) to control reasoning effort, an api_key field for each provider (instead of hardcoding), and a max_tokens field for response length control. The rules section gains an ast type for Tree-sitter queries (deferred to a future version) and a glob field for file matching with glob patterns. The workflow section gains an auto_approve field (none/writes/commands/all), a permission_mode field (ask/auto/smart), and a max_context_tokens field for conversation compaction thresholds. All v2.0 papyrus.yaml files work without modification in v3.0; the new fields default to safe values when absent.

10. Risk Assessment and Mitigations

Risk	Likelihood	Impact	Mitigation
Ink performance degrades with long conversations	Medium	High	Implement virtual scrolling (render only visible messages), compact old messages, benchmark with 1000+ message sessions
Streaming markdown parser edge cases	High	Medium	Adopt battle-tested markdown-it library with custom renderer, comprehensive test suite with real LLM output samples
Bun --compile produces large binaries	Medium	Low	Target 50MB max, use tree-shaking, lazy-load providers, offer npm install as alternative for size-sensitive users
API key security in distribution	Low	Critical	Never store keys in code, use environment variables and system keychain, warn on detected keys in config files
Terminal compatibility issues	Medium	Medium	Test on xterm-256color, tmux, screen, Kitty, iTerm2, Windows Terminal; provide fallback rendering for basic terminals
State file corruption on crash	Low	High	Write state atomically (temp file + rename), validate JSON on load, keep backup of previous state file

Table 11. Risk Assessment Matrix

11. Dependencies

Package	Version	Purpose	Criticality
ink	v5+	React renderer for the terminal	Core (cannot substitute)
react	v19+	Component model and hooks	Core (cannot substitute)
@anthropic-ai/sdk	latest	Claude API client with streaming	High (primary provider)
openai	v4+	OpenAI API client with streaming	High (secondary provider)
chalk	v5+	Terminal string styling (colors, bold)	Medium (cosmetic)
yaml	v2+	YAML parsing for papyrus.yaml	High (config loading)
diff	v7+	Text diff computation for DiffViewer	High (build visualization)
glob	v10+	File pattern matching	Medium (tool implementation)
marked	v14+	Markdown parsing	High (message rendering)
yargs	v17+	CLI argument parsing	Medium (can use raw process.argv)

Table 12. Core Dependencies

12. Conclusion

Papyrus v3.0 represents a complete transformation of the project from a functional but fragmented CLI tool into a polished, production-grade AI coding agent that provides the same immersive terminal experience as Claude Code, Qwen Code, and DeepSeek Coder. The key architectural decisions are: TypeScript with Bun as the unified language and runtime; Ink (React for the terminal) as the TUI framework; MCP-style stdio communication as the inter-process protocol; and a three-package monorepo structure separating concerns between the CLI entry point, the orchestration engine, and the terminal UI components.

What makes Papyrus unique among AI coding agents is not the TUI (which is replicated from Claude Code) but the orchestration engine underneath: the Plan-Audit-Build pipeline with deterministic rule enforcement, the auto-heal loop with three-tier retry strategies, and the state machine with crash recovery. These features, which originated in the v2.0 Go codebase and are being ported to TypeScript in v3.0, provide a layer of reliability and governance that no other AI coding agent offers. Claude Code can generate code; Papyrus can generate code reliably, within constraints, with automatic recovery from failures.

The eight-week implementation roadmap delivers a working product at each phase, starting with a basic chat interface in Week 2 and culminating in a fully-featured Claude Code-like experience with binary

distribution in Week 8. The migration strategy preserves backward compatibility with v2.0 configuration files while completely replacing the implementation. The reference projects (pi-mono and claude-canvas) have been carefully analyzed and their best patterns adopted without introducing unnecessary complexity. The result will be an open-source AI coding agent that combines the user experience of Claude Code with the engineering rigor of Papyrus.